

TECHNIKAI TUDÁSTÁR · MOBIL KIADÁS

Full Stack interjú — technikai Q&A

Redspher · Full Stack Developer

JS · TypeScript · React · Node.js · REST · SQL · AWS ·
AI / LLM / MCP · CI/CD · biztonság · rendszertervezés

2026. július · a kérdések fejezetenként számozva

Tartalom

1 · JavaScript-alapok	3
2 · TypeScript	9
3 · React	15
4 · Node.js	23
5 · REST és HTTP	29
6 · Adatbázisok és SQL	34
7 · AWS	38
8 · AI: LLM-ek, MCP, agentek	43
9 · Git, CI/CD, Docker	50
10 · Web-biztonság	53
11 · Tesztelés	55
12 · Rendszertervezés, logisztikai ízzel	57
13 · PHP és legacy-migráció	63
14 · Puska	65

Használat: kérdés → mondd ki hangosan a választ → csillagozd, ami bizonytalan, és arra menj vissza.

1 · JavaScript-alapok

TS/React-interjúk gyakran itt kezdenek. Ezeknek reflexből kell menniük.

1 · Mi a closure?

Függvény + a scope, amelyben definiálták: a belső függvény a külső változóit azután is eléri, hogy a külső lefutott.

- Klasszikus példa: számláló gyár — a visszaadott függvény „emlékszik” a `count`-ra.
- Erre épül a modulminta, a privát állapot, és a React hookok működése is.
- Csapda: `var` a ciklusban — minden callback ugyanazt az egy változót zárja be; `let` iterációként új kötést ad.

2 · Hogyan dől el a `this` értéke?

A hívás helyén, nem a definíciónál. Sorrend:

- `new` hívás → az új példány.
- `bind / call / apply` → az explicit megadott objektum.
- Metódushívás (`obj.fn()`) → a pont előtti objektum.
- Sima hívás → strict módban `undefined` (egyéb-ként a `global`).
- Arrow function: nincs saját `this`-e — lexikálisan a befoglalóét használja. Ezért jó callback-nek, és ezért rossz metódusnak/konstruktor-nak.

3 · Prototípus-lánc — és mi köze a `class`-hoz?

Minden objektumnak van prototípusa; a property-keresés a láncon megy felfelé, amíg találat vagy `null` nincs.

- A `class` szintaktikai cukor ugyanerre: a metódusok a `Klass.prototype`-ra kerülnek, az öröklés láncolás.
- Saját vs örökölt property: `Object.hasOwn(obj, key)` (ill. `hasOwnProperty`).

4 · `==` vs `===`?

A `==` típuskonverzióval hasonlít (meglepetésekkel: `"" == 0`), a `===` típus + érték szerint.

- Alapszabály: mindig `===`.
- Egyetlen elfogadott idióma: `x == null` — egyszerre igaz `null`-ra és `undefined`-re.

5 · `var` / `let` / `const`, hoisting, TDZ?

- `var`: függvény-scope, hoistolva `undefined`-ként — ezért „működik” deklaráció előtt, csendben hibásan.
- `let` / `const`: blokk-scope; deklarációig temporal dead zone — a korai hozzáférés hibát dob.
- Function declaration teljesen hoistolódik; a function expression / arrow nem.
- A `const` a kötetést védi, nem az objektum tartalmát.

6 · Promise-ok: állapotok és kombinátorok?

Állapotok: pending → fulfilled vagy rejected (végleges). A `.then` láncol, a hiba a láncon végigesik az első `.catch`-ig.

- `Promise.all` — mind kell; az első hiba buktatja (fail-fast).
- `Promise.allSettled` — mindegyik kimenetele kell, hiba nem szakít.
- `Promise.race` — az első, ami eldől (timeout-mintához).
- `Promise.any` — az első sikeres.

7 · `async/await` — mire figyelsz?

Cukor a promise-ok fölött: az `await` felfüggeszti a függvényt, a hibák `try/catch`-cel kezelhetők.

- Soros vs párhuzamos: `await` ciklusban egyenként vár; független munkára `Promise.all`.
- Nagy listánál korlátozott konkurrencia (pl. `p-limit`), hogy ne dőljön rá a downstreamre.
- Ne felejts `await`-et: a „lebegő” promise hibája `unhandled rejection` lesz.

8 · Böngésző event loop: task vs micro-task?

Minden task (pl. `setTimeout`-callback, I/O) után a teljes microtask-sor kiürül (promise-callbackek, `queueMicrotask`), csak utána jöhet render vagy a következő task.

- Ezért fut le a `Promise.resolve().then(...)` a `setTimeout(..., 0)` előtt.
- Végtelen microtask-lánc kiéhezteti a rendere-
lést.
- A Node-oldali fázisokat lásd a 4. fejezetben.

9 · Shallow vs deep copy, immutabilitás?

- Spread / `Object.assign` / `Array.prototype.slice`: sekély — a beágyazott objektumok közösek maradnak.
- Mély másolat: `structuredClone` (függvényeket, DOM-ot nem visz).
- Immutábilis frissítés (új objektum a régi módosítása helyett) — a React-diffelés és az időuta-zó debug alapja; mélyen fészkeltnél Immer-jellegű segítség.

10 · ESM vs CommonJS?

- ESM: `import/export` , statikus elemzés → tree-shaking, top-level await; a modern default.
- CJS: `require/module.exports` , futásidejű, szinkron — a Node-örökség.
- Node-interop: `"type": "module"` vagy `.mjs / .cjs` ; ESM-ből CJS importálható, fordítva csak dinamikus `import()` -tal.
- Könyvtárszerzőként dual package (exports mező) — és a belőle fakadó „két példány” csapda.

11 · Debounce vs throttle?

- Debounce: csak az utolsó esemény után X idővel fut — keresőmező, átméretezés vége.
- Throttle: legfeljebb X időnként fut — scroll- és pozíciókezelők.
- Említhető: leading/trailing variánsok, és hogy Reactben ezt ref-ben tartott, stabil függvénnnyel csinálod.

12 · Map/Set vs objektum/tömb — mikor melyik?

- `Map` : bármilyen kulcstípus, garantált beszúrási sorrend, `size`, gyors gyakori ki-be pakolás — objektum helyett „szótárnak”.
- `Set` : egyediség és $O(1)$ tartalmazás-teszt — dedup: `[...new Set(arr)]`.
- `WeakMap` / `WeakSet` : a kulcs GC-zhető marad — metaadat objektumokhoz szivárgás nélkül.

13 · Mitől „first-class” a függvény JS-ben, és mi a magasabb rendű függvény?

- A függvény érték: változóba tehető, átadható, visszaadható — erre épül a `map/filter/reduce`, a `middleware`-ek és a `hookok` világa.
- Magasabb rendű: függvényt kap vagy ad vissza (pl. egy `withRetry(fn)` wrapper).
- Curry/partial application: paraméterek előre rögzítése — konfigurált segédfüggvényekhez.

2 · TypeScript

A hirdetés kemény léce. Rövid definíció + egy konkrét példa minden válaszban.

1 · Miért TypeScript egy üzleti terméken, a „kevesebb bug”-on túl?

A típus futtatható dokumentáció és refaktorálási védőháló.

- Mező-átnevezés, API-kontraktus-változás: fordító által vezetett feladat grep-és-imádkozás helyett.
- Üzleti szabályok a típusrendszerben: `OrderStatus` union — az érvénytelen állapot le sem fordul.
- De: a típusok futásidőben törölődnek → minden határon (HTTP, queue, env) runtime-validálás (Zod), a statikus típus a sémából származtatva — egy igazságforrás.

2 · `interface` vagy `type`?

- Objektum-alakra nagyrészt felcserélhetők.
- `interface` : declaration merging, idiomatikus publikus/bővíthető kontraktusokhoz.
- `type` : union, metszet, mapped/conditional típus, tuple — ezekhez kötelező.
- Ökölszabály: `interface` objektum-kontraktsra, `type` alias minden másra — és egy konzisztens csapatkonvenció.

3 · any VS unknown VS never ?

- `any` : mindkét irányban kikapcsolt fordító; megfertőz mindent — csak karanténózott határon.
- `unknown` : biztonságos top type — bármi ráadható, de használat előtt szűkíteni kell; külső input helyes típusa.
- `never` : üres típus — elérhetetlen kód, kimerítő switch maradéka.

4 · Mutass generikust constrainttel.

```
function pluck<T, K extends keyof T>(
  items: T[], key: K
): T[K][] {
  return items.map(i => i[key]);
}
// pluck(orders, "status") -> OrderStatus[]
```

A `K extends keyof T` a kulcsot a valódi property-khez köti: az elgépelés fordítási hiba, a visszatérési típus pontosan következtetett.

5 · Mely utility type-okat használsd ténylegesen?

- `Partial` / `Required` — patch payloadok.
- `Pick` / `Omit` — DTO a domain-modellből.
- `Record<K,V>` — kulcsolt map; `Readonly` — imutábilis nézet.
- `ReturnType` / `Parameters` / `Awaited` — wrappelés, promise-kibontás.
- Elv: egy kanonikus alakból származtass, ne tarts kézzel másolt duplikátumokat.

6 · Diszkriminált unionok — miért annyira hasznosak?

```
type ShipmentEvent =  
  | { type: "picked_up"; at: string }  
  | { type: "delayed"; at: string; reason:  
    string }  
  | { type: "delivered"; at: string; pod:  
    string };  
  
switch (e.type) {  
  case "picked_up": ...  
  case "delayed":    ...  
  case "delivered": ...  
  default: { const x: never = e; return x; }  
}
```

- A `type` tag minden ágban automatikusan szűkít.
- A `never` default kimerítővé teszi: új variáns → minden lekezeletlen switch fordítási hiba.
- Eseményfolyamok (pl. szállítmány-státuszok) természetes modellje.

7 · Type guardok és szűkítés (narrowing)?

- Beépített: `typeof`, `instanceof`, `in`, diszkriminációs-check.
- Egyéni guard: `value is X` visszatérés; assertion function: `asserts value is X` — false helyett dob.
- Csapda: a rossz kézi guard hazudik a fordítónak → külső adatra séma-validálás (Zod `safeParse`): szűkít és ellenőriz.

8 • `satisfies` vs `as` ?

- `as` : assertion — felülbírálja a fordítót, valódi hibát rejthet el; csak ott, ahol tényleg többet tudsz nála (+ `as const`).
- `satisfies` : ellenőrzi az alakot, *de megőrzi a szűkebb következtetett típust* — `config`-objektumok, `lookup-táblák` ideális eszköze.
- Példa: `const config = {...} satisfies AppConfig` — validált alak, literál típusok az `autocomplete`-hez.

9 • Mely `strict` flagek számítanak?

- `strict: true` az alap; a lelke a `strictNullChecks` — a `null`-hibák fordításkor buknak.
- `noUncheckedIndexedAccess` : `index`-hozzáférés `T | undefined` — a hiányzó elem kezelése kikényszerítve.
- `exactOptionalPropertyTypes` : az „opcionális” és az „explicit undefined” szétválasztása.

10 · A TS strukturálisan típusos — következmények?

A kompatibilitás alak szerinti, nem név szerinti: bármely objektum megfelel, amelynek stimmelnek a tagjai.

- Kockázat: véletlen egybeesés — `CustomerId` vs `OrderId`, mindkettő `string`.
- Ellenszer: `excess-property check` literálokon;
branded type (`type OrderId = string & { __brand: "OrderId" }`) ott, ahol az ID-keverés valódi üzleti bug.

11 · Enum vagy union?

- `String`-literál `union` (`"pending" | "paid"`): nincs `runtime`-kód, `JSON`-nal barát, jól szűkít — a tipikus default.
- `enum`: futásidejű objektum is; `const enum` `inline`-olódik, de `tooling`-korlátokkal.
- Gyakori minta: `const STATUSES = [...]` as `const + typeof STATUSES[number]` — érték és típus egy forrásból.

12 · Hogyan migrálnál JS-kódbázist TS-re?

- `allowJs` + `checkJs`, majd fájlankénti konverzió a legtöbbet importált moduloktól.
- `.d.ts` a típusozatlan belsőkhöz; a szigorúság flagenként felhúzva (racsni, nem big bang).
- CI-kapu: „nincs új `any`”; a kódbázis végig szállítható marad.
- Ugyanez a minta legacy rendszer típusos API-kontraktus mögé csomagolására is.

13 · Hogyan maradnak szinkronban az API-típusok frontend és backend között?

- Egy igazságforrás, kifelé generálva: közös csomagban Zod-sémák (runtime + statikus típus egyből), vagy OpenAPI generált kliensekkel.
- Amit kerülsz: kézzel duplikált interface-ek két oldalon — driftelnek, és a fordító nem lát át a HTTP-résen.

3 · React

A válaszokat a renderelési modellhez horgonyozd — abból minden más levezethető.

1 · Vezess végig a React renderelési modellen.

State/props változás → újrender (alapból a teljes al-fa) → új elemfa → rekonsziliáció diffeli az előzővel → csak a valódi különbség kerül a DOM-ba.

- A key stabil identitást ad listaelemeknek: state és DOM-node újrahasznosul.
- Következmény 1: a render legyen tiszta — többször is lefuthat.
- Következmény 2: a teljesítmény-munka = state lejjebb mozgatása vagy határok memoizálása, nem DOM-bűvészkedés.

2 · `useEffect` — mikor és hogyan helyes?

Effect = szinkronizálás egy *külső* rendszerrel (feliratkozás, timer, imperatív API).

- Kimerítő dependency-lista (lint-szabály bekapcsolva); cleanup mindenhez, ami feliratkozik.
- A StrictMode dupla mount-unmountja dev-ben: ingyen teszt, hogy a cleanup jó.
- Amire nem való: származtatott state (számod render közben), user-esemény (handlerben), effect-lánccal state-frissítés.

3 · Mikor segít ténylegesen a `useMemo` / `useCallback` / `React.memo`?

Mind a referenciális egyenlőségről szól.

- `React.memo` : kihagyja a gyerek-renderét, ha a propok shallow-egyenlők — de ehhez a bemenő objektum/callback identitása legyen stabil: `useMemo` / `useCallback` .
- Ezen túl: tényleg drága számítás, effect-dependency stabilizálás.
- Nem ingyenes (memória + összehasonlítás), és az idő előtti szórás strukturális bajt rejt el. Sorrend: mérés → state-átrendezés → memo a bizonyított forró határookra.

4 · State-kezelés egy komoly React-app-ban?

- Első vágás: **szerver-state** $\neq$ **kliens-state**.
- Szerver-state (rendelések, ajánlatok): TanStack Query-féle cache — dedup, háttér-refetch, invalidálás, optimista frissítés készen.
- Kliens/UI-state: lokálisan, ameddig lehet; megosztásra valami könnyű (Zustand) vagy Redux Toolkit, ha a csapat azon áll.
- Context: alacsony frekvenciájú globálok (téma, auth-identitás) — nem state-manager.

5 · Mi a Context csapdája?

- Minden consumer újrenderel az érték változásakor — és az inline `value={{...}}` minden rendernél új.
- Ellenszerek: memoizált érték; egy kövér context szétvágása (state vs dispatch); gyakori frissítésre selectoros store.
- Egy mondatban: a Context terjeszt, nem optimalizál.

6 · Controlled vs uncontrolled inputok, űrlapok?

- Controlled: React state az igazságforrás (`value` + `onChange`) — teljes kontroll, minden leütés render.
- Uncontrolled: a DOM tartja az értéket, ref-fel olvasod.
- Üzleti űrlapra: react-hook-form — belül uncontrolled (gyors), Zod-resolveres séma-validálással, így űrlapszabály és API-kontraktus egy definíció.

7 · Refek — mire valók (és mire nem)?

- `useRef` : renderek közt megmaradó, változtatható tároló, aminek a módosítása nem rendel — DOM-elérés (fókusz, mérés, scroll), timer-ID-k, „előző érték”.
- Komponensen átadva: régebben `forwardRef` , React 19-től a ref sima prop.
- Nem való: render közbeni írás/olvasás vezérlésre — az state; a ref nem triggerel frissítést.

8 · Mitől jó egy custom hook?

- Állapotos logika újrafelhasználása UI nélkül: `useDebounce` , `useShipmentTracking` .
- `use` prefix + a hook-szabályok öröklődnek (top level, feltétel nélkül).
- Tiszta kontraktus: mit kap, mit ad vissza; a komponens deklaratív marad, a logika külön tesztelhető.

9 · Code splitting és hibakezelés a UI-ban?

- Route-szintű splitting: `React.lazy` + `Suspense` ; nehéz widgetek (chart, térkép) külön.
- Error boundary route/feature-héjakon: egy összeomló widget fallbackre degradál, nem fehér képernyő.
- Párosítsd hibariportolással — a hiba legyen látható, ne elnyelt.

10 · SSR, hydration, Server Components — mi mit jelent?

- SSR: a HTML a szerveren áll elő → gyors első tartalom, SEO.
- Hydration: a kliens ráköti az eseménykezelőket a kész HTML-re; mismatch-hiba oka tipikusan dátum/random/böngésző-only API a renderben.
- RSC: a szerver-komponens kódja nem megy le a kliensre, adatot közvetlenül olvashat; kliens-komponens kell interakcióhoz. Frameworkök (pl. Next.js App Router) adják a keretet.

11 · Mit hozott a React 18/19, amit használsz?

- 18: automatikus batching mindenhol; `useTransition` / `useDeferredValue` a reszponzív gépeléshez drága listák mellett; `useId` ; érő `Suspense`.
- 19: form Actionök + `useActionState` / `useOptimistic` (kevesebb kézi pending/error state), `use()` , ref mint sima prop, RSC mainstream.

12 · Lassú egy oldal — a debug-meneted?

- Reprodukálás → React DevTools Profiler: mi renderel és miért (mely prop/state változott).
- Tipikus tettesek: túl magasan élő state; instabil objektum/callback-propok; óriási lista virtualizálás nélkül; layout-thrash effectből.
- Javítás sorrendje: strukturális (state lejjebb, bontás, react-window) → memo a forró útra → újramérés.
- Betöltésre: bundle-analízis, code splitting, cache-headerek a mikrooptimalizálás előtt.

13 · Gyakori React anti-patternek review-ban?

- Tömbindex key-ként átrendezhető listán — state átvérzés.
- Prop másolása state-be („tükör-state”) — azonnali elavulás; származtasd.
- Effect-láncok state-frissítéssel — adatfolyam-átstrukturálás kell.
- 800 soros isten-komponens (fetch + form + táblázat) — bontás felelősség szerint.
- JSX-be temetett üzleti logika — hookba/tiszta függvénybe, hogy tesztelhető legyen.

14 · Hogyan tesztelsz Reactet?

- React Testing Library, felhasználó-szemszög: role/label szerinti query, interakció, látható ki-menet — nem implementációs részlet, így a refaktor nem töri a suite-ot.
- Hálózat a határon mockolva (MSW) — a kom-ponens a valódi fetch-útját futtatja.
- Kiemelt hookok/logika: közvetlen unit-teszt; vékony E2E (Playwright) a pénzes utakra.

15 · TypeScript-minták Reactben?

- Variáns-propok diszkriminált unionnal: `{ kind: "link"; href }` VS `{ kind: "button"; onClick }` — érvénytelen kombináció le sem fordul.
- Generikus komponens: `<Table<Order>>` táblá-zatra/selectre.
- `ComponentProps<typeof X>` meglévő komponens bővítésére.
- Szerver-state típusok a közös API-sémából kö-vetkeztetve, nem újradeklarálva.

16 · Mikor virtualizálsz listát, és hogyan?

- Ha a DOM-elemszám ezres nagyságrendbe megy (rendeléslista, eseménynapló): csak a látható ablak renderelődik (react-window / TanStack Virtual).
- Feltételek: ismert/mérhető sormagasság, stabil key-k; a fókusz- és görgetéspozíció kezelése.
- Előtte olcsóbb kérdés: kell-e egyáltalán ennyi elem — lapozás/szűrés szerveroldalon.

4 · Node.js

Az event loop a belépő; a többi kérdés jellemzően üzemeltetési gyakorlatot mér.

1 · Magyarázd el a Node event loopot.

A JS egy szálon fut; a libuv event loopja fázisokban dolgozza fel a callbackeket.

- Fázisok: **timers** → pending callbacks → **poll** (I/O) → **check** (`setImmediate`) → close.
- Callbackek között a microtaskok teljesen kiürülnek: előbb `process.nextTick`, aztán a promise-callbackek.
- Az I/O az OS-nél / a libuv thread pooljában fut (fs, DNS, crypto) — az „egyszálú” csak a te JS-edre igaz.
- Következmény: CPU-nehéz szinkron munka *minden* kérést befagyaszt — ki kell szervezni.

2 · Hogyan skálázod a Node-ot, és hova teszed a CPU-igényes munkát?

- Processzen belül: nem blokkoló handlererek; CPU-munka (PDF-generálás, crypto, óriásparsolás) → `worker_threads` vagy külön szolgáltatás/queue.
- Gépen/klaszteren: konténerenként egy processz, horizontális skálázás load balancer + health checkek mögött (a cluster/PM2 a konténer előtti világ idiómája).
- Állapot kifelé (Redis) — sticky session csak végszükségben.

3 • `worker_threads` VS `child_process` ?

- Worker: ugyanabban a processzban, közös memória lehetséges (`SharedArrayBuffer`), olcsóbb — CPU-nehéz JS-hez.
- Child process: külön processz, teljes izoláció, bármilyen program futtatható — külső binárishoz, sandboxhoz.
- Mindkettőnél: pool-ozd, ne kérezenkénti indítás.

4 • Streamek és **backpressure** — mikor számítanak?

- Amikor az adat nagyobb a memóriánál vagy időben érkezik: több milliós CSV-export, feltöltés proxyzása S3-ba, log-transzformálás.
- A stream chunkonként, korlátos memóriával dolgozik; a **backpressure** a flow control: a lassú fogyasztó megállítja a termelőt, nem nő a buffer a végtelenbe.
- `stream.pipeline`-t használj (vagy web streamet): hibát és cleanupot helyesen terjeszt — a kézi `.pipe()` lánc híresen nem.

5 · Hibakezelési stratégia egy Node-szolgáltatásban?

- try/catch értelmes határokon; központi error middleware: ismert hibaosztály → HTTP problem-válasz, minden más → logolt 500 correlation ID-vel.
- Operational hiba (várható, retryelhető) ≠ programozói hiba (crash-t érdemel).
- `unhandledRejection` / `uncaughtException` : logol és kilép — a félig törött processz rosszabb az újraindítotttnál.
- Graceful shutdown SIGTERM-re: új kapcsolat stop → in-flight kifuttatás → poolok zárása → exit; zero-downtime deploy alapja.

6 · Express vs Fastify vs NestJS?

- Express: minimális, óriási ökoszisztéma — minden fegyelmen múló konvenció.
- Fastify: gyors, séma-alapú validálás/szerializálás beépítve — karcsú szolgáltatás defaultja.
- NestJS: véleményes struktúra (modulok, DI, guardok, interceptorok) — növekvő csapatnál/kódbázisnál térül meg.
- Lényeg: a framework kevésbé számít, mint a határok, amiket kikényszerítesz vele (validálás, rétegek, kontraktusok).

7 · Konfiguráció és secretok?

- Twelve-factor: config a környezetből, sosem commitolva.
- Env-séma (Zod) bootkor: a hiányzó változó induláskor bukik, nem hajnali 3-kor egy request-ben.
- Secretok a platform secret store-jából (AWS Secrets Manager / SSM, K8s secret), futásidőben injektálva; lokálisan gitignore-olt `.env`.
- Ugyanaz az artifact megy át környezeteken, csak a config változik.

8 · API-biztonsági lista, amit ténylegesen betartasz?

- Minden input séma-validálva a peremen — injection és type-confusion itt hal meg.
- AuthN $\neq$ AuthZ: JWT-ellenőrzés *után* erőforrás-szintű jogosultság (IDOR az #1 valós lyuk).
- Rate limiting, payload-méretkorlát, helmet-jellegű headerek.
- Paraméterezett query / ORM — soha stringből épített SQL.
- Függőség-higiénia: lockfile, `npm audit` / Dependabot CI-ban, minimális base image.
- Se secret, se PII a logokban (GDPR).

9 · Hogy néz ki az observability egy szolgáltatásodban?

- Strukturált JSON-log (pino) correlation ID-vel minden ugráson át — ebből áll össze egy kérés története.
- Metrikák a négy arany jelre (latencia, forgalom, hiba, telítettség); riasztás user-tünetre (p95, hibaarány), nem CPU-hiúságra.
- Liveness/readiness endpointok — a platform újraindít és kikerül.
- Distributed tracing: trace ID + spanek szolgáltatásokon át (OpenTelemetry a de facto szabvány).

10 · Lassú egy endpoint — végigmész rajta?

- Előbb mérés: event loop blokkolt (loop lag)? downstream lassú (DB/HTTP időzítés a logból)? N+1?
- Javítások gyakorisági sorrendben: hiányzó index / N+1 batchelése → cache TTL-lel és invalidálási szabállyal → connection pooling → streamelés bufferelés helyett → CPU-munka kiszervezése.
- Ha tényleg CPU: profilozás (`clinic`, `0x`, `inspector`). A nyereséget load-teszt rögzíti.

11 · Memóriaszivárgás Node-ban — okok és diagnózis?

- Tünet: folyamatosan növő heap/RSS, GC-idő nő, végül OOM-restart.
- Tipikus okok: korlátlan in-memory cache/map; el nem távolított event listenerek; closure-ben fogva tartott nagy objektumok; ottfelejtett timererek.
- Diagnózis: heap snapshotok összevetése (inspector/Chrome DevTools), retainer-lánc; teherpróbával reprodukálva.
- Megelőzés: méretkorlátos/TTL-es cache (LRU), WeakMap metaadatra, cleanup mindenhez.

5 · REST és HTTP

Logisztikai platform = API-k és webhookok. Az idempotencia a sztártéma — hozd fel magadtól.

1 · Hogyan tervezel erőforrás-orientált API-t?

- Főnevek az erőforrásokra: `/shipments`, `/shipments/{id}/events`; HTTP-igék a műveletekre; többes számú kollekciók.
- Ami nem CRUD, az explicit akció-erőforrás: `POST /shipments/{id}/cancel` — pragmatizmus a purizmus előtt.
- A konzisztencia veri az okosságot: egységes nevezéktan, lapozás, hibaalak mindenhol.

2 · Milyen státuszkódokra és hibaformára standardizálsz?

- Siker: 200 olvasás · 201 létrehozva (Location) · 202 async elfogadva · 204 nincs body.
- Kliens: 400 hibás · 401 nem azonosított · 403 nincs joga · 404 nincs · 409 állapot-konfliktus („kiszállított nem mondható le”) · 422 szemantikai hiba · 429 throttle.
- 5xx = a mi hibánk.
- Hiba-boríték: RFC 7807 problem+json — gépi kód + emberi üzenet + correlation ID; stack trace soha.

3 · Idempotencia — magyarázd el rendesen.

Idempotens művelet kétszer = ugyanaz, mint egyszer. GET/PUT/DELETE kontraktus szerint az; a POST nem — pedig a hálózat retryel.

- Létrehozó műveletekre `Idempotency-Key` header: első kérés eltárolja kulcs + választ; az azonos kulcsú retry a tároltat kapja — nincs dupla foglalás.
- Ugyanez lejjebb: `at-least-once` queue-k és webhookok fogyasztói event ID szerint deduplikálnak.
- Fuvarozásban a duplikált foglalás valódi pénz — ezért sztárkérdés.

4 · PUT vs PATCH?

- PUT: a teljes erőforrás cseréje — idempotens definíció szerint.
- PATCH: részleges módosítás; önmagában nem garantáltan idempotens — tedd azzá (abszolút értékek, verzió-feltétel).
- Konkurencia: optimista zár `If-Match` + `ETag` — az elavult írás 412-t kap.

5 • Lapozás — melyiket és miért?

- Cursor-alapú (átlátszatlan cursor az utolsó elem rendezőkulcsából): stabil insertek alatt, $O(1)$ bármely mélységben — minden nagy/forró listára.
- Offset/limit: csak kis admin-listákra — a mély OFFSET table scan, és a sorok elcsúsznak.
- Mindig: `next_cursor` a válaszban, serveroldali lapméret-plafon.

6 • API-verziózás?

- Additive-first: opcionális mező hozzáadása nem verzióemelés; a kliens ignorálja az ismeretlen mezőt.
- Törő változásra `/v2` az útvonalban — látható, cache-elhető —, párhuzamos futás, deprecation-ablak, használati metrikák.
- Mélyebb szint: ez kontraktus-menedzsment — OpenAPI + generált kliensek + contract-tesztek elkapják a törést az ügyfél előtt.

7 · AuthN/AuthZ API-khoz?

- OIDC/OAuth2 identity providerrel: authorization-code + PKCE böngészőnek, client-credentials service-to-service-nek.
- Rövid életű JWT access token — aláírás/issuer/audience/lejárát validálva; refresh-token rotációval.
- A claim identitás + durva szerep; a finom jogosultság szerveroldalon, az erőforrás ellen („az ő szervezetéé-e ez a szállítmány?”) — a token elavul, az IDOR a klasszikus rés.

8 · Tervezz webhook-rendszert partnerek felé és felől.

- Kimenő: HMAC-aláírt payload + timestamp (replay ellen); boríték egyedi event ID + típus + verzió; retry exponenciális backoffal + jitter-rel 2xx-ig; dead-letter lista + újraküldő felület.
- Bejövő: aláírás-ellenőrzés → gyors 2xx → aszinkron feldolgozás queue-ból; dedup event ID szerint (at-least-once!).
- Az eseményeket dokumentáld API-ként — mert azok.

9 · HTTP-cache-elés — mit használsz belőle?

- Statikus assetek: fájlnevében hash + Cache-Control: immutable, max-age=1év — a deploy cseréli az URL-t.
- API-válaszok: ETag + If-None-Match → 304, sávszélesség-spórolás; óvatos max-age / stale-while-revalidate.
- private vs public: user-specifikus válasz CDN-re nem való.
- A cache-kulcs része a vary (pl. Accept-Language) — elfelejtve keresztbe-cache-elés.

10 · Mikor rossz eszköz a REST?

- Hosszan futó munka → 202 + státusz-erőforrás / job queue, nem 90 mp-es request.
- Valós idejű push (követés) → SSE vagy WebSocket.
- Nagy volumenű belső service-to-service → gRPC vagy aszinkron üzenetkezelés.
- Aggregáció-nehéz UI → BFF-réteg, hogy a mobil ne hét hívást intézzon.
- A szenior válasz interakciós mintánként választ, nem stílushoz hűségés.

6 · Adatbázisok és SQL

Full-stack interjún szinte garantált blokk. Postgres-fókusz.

1 · Hogyan működik egy index, és mikor segít?

- Default: B-tree — rendezett kulcsstruktúra, $O(\log n)$ keresés; segít WHERE/JOIN/ORDER BY szelektív oszlopain.
- Composite index: bal-prefix szabály — `(customer_id, created_at)` jó a `customer_id`-ra és a párosra, önmagában a `created_at`-ra nem.
- Covering index (INCLUDE): a query az indexből kiszolgálható, heap-olvasás nélkül.
- Ára: minden írás karbantartja + hely; alacsony szelektivitású oszlopra (pl. boolean) önmagában értelmetlen.

2 · EXPLAIN (ANALYZE) — mit nézel benne?

- ANALYZE ténylegesen lefuttatja → valós idők és sorszámok.
- Piros zászlók: Seq Scan nagy táblán szelektív feltétel mellett; a becsült vs valós sorszám nagy eltérése (elavult statisztika → ANALYZE); nested loop óriási halmazokon; lemezre csorduló sort/hash.
- Munkamenet: leglassabb node megkeresése → hiányzó index / átírt feltétel → újramérés.

3 · Tranzakciók és ACID?

- Atomicity: mind vagy semmi. Consistency: constraintek a tranzakció végén is állnak. Isolation: párhuzamos tranzakciók nem látják egymás félkész állapotát (szint szerint). Durability: commit után áramszünet-álló.
- Gyakorlat: a tranzakció legyen rövid; ne tarts benne külső hívást (HTTP) — lock-idő és összeakadás.

4 · Izolációs szintek és anomáliák?

- Anomáliák: dirty read (más commitálatlan adata), non-repeatable read (ugyanaz a sor kétszer másképp), phantom (új sorok jelennek meg a feltételre).
- Postgres default: Read Committed — utasításoként friss snapshot.
- Repeatable Read: tranzakció-szintű snapshot. Serializable (PG: SSI): mintha sorban futnának — szerializációs hibára retry-logika kell.
- Ökölszabály: Read Committed + explicit zárok/feltételes update-ek a kritikus pontokon; Serializable ott, ahol az invariáns tényleg megköveteli.

5 · Mi az N+1 probléma, és hogyan szünteted meg?

- 1 lista-query + elemenként 1 kapcsolódó query (ORM lazy loading) → 1+N kör az adatbázisba.
- Felismerés: query-log / APM — sok azonos alakú kis query.
- Megoldás: JOIN vagy `WHERE id IN (...)` batch; ORM-ben eager loading (include/rerelations); GraphQL-nél dataloader.

6 · Optimista vs pesszimista zárolás?

- Optimista: version oszlop + feltételes UPDATE (`WHERE id=? AND version=?`); ha 0 sor módosult, valaki megelőzött → újratöltés/retry. Olcsó, ha ritka az ütközés.
- Pesszimista: `SELECT ... FOR UPDATE` — a sor zárolva a tranzakció végéig; ütközés-sűrű kritikus szakaszra.
- Deadlock ellen: konzisztens zárolási sorrend, rövid tranzakciók, retry a deadlock-hibára.

7 · Normalizálás vs denormalizálás?

- OLTP-alap: ~3NF — nincs duplikáció, nincs update-anomália.
- Denormalizálás tudatosan, olvasás-nehéz utakra: cache-elt aggregátum, materialized view, előszámolt oszlop — a szinkronban tartás költségével.
- A helyes sorrend: normalizált mag + mért igény alapján denormalizált olvasati réteg.

8 · Connection pooling — miért és hogyan?

- A DB-kapcsolat drága (kézfogás, memória, Postgresnél processz) — poolból újrahasznosított.
- A pool mérete véges erőforrás: összehangolva a DB `max_connections`-szel; sok példány × nagy pool = kimerített adatbázis.
- Sok kliens / serverless esetén külső pooler (PgBouncer, RDS Proxy).
- Tünetek, ha rossz: „too many connections”, várakozás poolra p95-csúcsokkal.

7 · AWS

A pozícióhoz tartozó serverless/queue-alapkészlet, döntési szempontokkal.

1 · Lambda vs Fargate vs EC2 — hogyan választasz?

- **Lambda**: tüskés/eseményvezérelt munka, webhook-handlerek, glue — nulla üresjáratnyi költség, nullára skálázódik; de 15 perces plafon, cold start, és tartós nagy terhelésen megforduló árazás.
- **Fargate (ECS)**: hosszú életű szolgáltatás, WebSocket, egyenletes forgalom — konténer node-menedzsment nélkül.
- **EC2**: speciális hardver / maximális kontroll — patchelés és kapacitás-menedzsment árán.
- Döntési inputok: forgalom alakja, futásidő, latencia-érzékenység, teljes költség a várt terhelésen.

2 · DynamoDB vagy relációs adatbázis?

- DynamoDB: ismert, kulccsal címezhető access patternök — session state, eseményfolyam, nagy skálájú lookup; partition/sort key a query-k köré tervezve, GSI a másodlagos mintákra.
- Relációs (RDS/Aurora Postgres): ad-hoc query, több-entitásos tranzakció, riporting, constraintek — a legtöbb üzleti mag ide tartozik.
- Pragmatikus default: Postgres a system of record, Dynamo/Redis a kinőtt forró utakra.
- Dynamóban a query-t modelled előbb, az adatot utána — a normalizálási ösztön fordítottja.

3 · SQS-mechanikák, amiket tudni kell?

- At-least-once → a fogyasztó legyen idempotens.
- Visibility timeout: az in-flight üzenet rejtve; törlés nélküli crash után újra megjelenik — a timeout legyen a legrosszabb feldolgozási idő fölött, hosszú jobnál heartbeat-hosszabbítás.
- DLQ N sikertelen receive után + riasztás a mélységére — poison-message védőháló.
- Standard: nagy áteresztés, néha duplikátum, laza sorrend; FIFO: sorrend + dedup üzenet-csoportonként, kisebb áteresztés.

4 · SNS vs EventBridge?

- SNS: egyszerű pub/sub fan-out; klasszikus tartósság-minta: SNS → SQS fogyasztónként.
- EventBridge: esemény-busz tartalom-alapú routing-szabályokkal, schema registryvel, natív SaaS/AWS forrásokkal — szolgáltatásokon átívelő eseményvezérelt architektúrához.
- A közös pont a lényeg: termelő és fogyasztó szétcsatolása.

5 · S3-minták, amiket egy full-stack fejlesztő rendszeresen használ?

- Presigned URL: a böngésző közvetlenül tölt fel/le rövid életű aláírt URL-lel — a fájl nem folyik át az API-n.
- Event notification (S3 → Lambda/SQS): feldolgozás triggerelése feltöltéskor (POD-fotó, fuvarokmány).
- Lifecycle-szabályok olcsóbb tárolási szintekre; CloudFront a publikus assetek elé.
- Public access alapból blokkolva; hozzáférés IAM role-okon át.

6 · IAM egy percben?

- Identitások policy-t kapnak; a szolgáltatások **role**-t vesznek fel — hosszú életű access key az appban: soha.
- Least privilege: ez a Lambda ezt a bucket-prefixet olvassa, erre a queue-ra ír — semmi több.
- Resource policy (bucket/queue oldal) kiegészíti az identity policy-t; az explicit deny mindig nyer.
- Gyakorlat: szolgáltatásonkénti role IaC-ben, platform által injektált, rotált credentialök.

7 · A Lambda üzemeltetési éles sarkai?

- Cold start — kis bundle (esbuild), latencia-kritikus útra provisioned concurrency.
- 15 perces plafon; payload- és /tmp-limitek.
- Concurrency-limit: throttle-kockázat és védő-eszköz is (reserved concurrency egy törékeny downstream DB elé).
- Init-kód a handleren kívül → kapcsolat-újra-hasznosítás; async invoke-ra DLQ/failure destination.
- Az invokációnkénti strukturált CloudWatch-log a fekete dobozod.

8 · IaC és deploy AWS-re — mivel és hogyan?

- Minden reprodukálható kódból; TS-csapatnak a CDK természetes: típusos infrastruktúra, code review, pipeline-ból deploy, diff az apply előtt.
- Alternatívák: Terraform (multi-cloud, deklaratív HCL), SAM (tisztán serverless).
- Elv ugyanaz, mint GitOpsnál: a git az igazságforrás, a környezet hozzá konvergál, a rollback egy revert.

Vázolható mini-architektúra: partner-webhook fogadás AWS-en

API Gateway → vékony Lambda: HMAC-ellenőrzés, érvénytelen eldobása, nyers esemény **SQS**-be (200-as válasz <100 ms) → fogyasztó (Lambda/Fargate): séma-validálás, **dedup event ID szerint**, upsert Postgresbe, domain-esemény **EventBridge**-re → hibák backoffal retryelnek → poison üzenet **DLQ**-ba, riasztással. Minden ügrásnál idempotens, burst ellen pufferelt, szakaszonként külön skálázható — a trade-offokat hangosan mondd.

8 · **AI: LLM-ek, MCP, agentek**

A hirdetés explicit témája. Trade-offokban beszélj, ne hype-ban.

1 · **Hogyan működik egy LLM — két szinten?**

- Technikai: transformer, ami hatalmas korpuszon tanulva a következő tokent jósolja; a viselkedést utótanítás (instrukciókövetés, RLHF) formálja; csak a kontextusablakát „látja”.
- Üzleti megfogalmazás: nagyon jó nyelv transzformálásában — összefoglal, kinyer, megfogalmaz —, de valószínűségi, és képes folyékonyan tévedni.
- Ezért a tervezési minta: jó kontextus be, korlátozott kimenet, validálás, ember a kockázatos döntéseken.

2 · **Milyen gyakorlati eszközeid vannak LLM-integrációnál?**

- System prompt szereppel, szabályokkal, kimeneti kontraktussal; few-shot példák formátum-érzékeny feladatra.
- Alacsony temperature extrakcióra/klasszifikációra; magasabb csak kreatív szövegre.
- Strukturált kimenet (JSON-séma / tool-séma) → megbízható parszolás; utána Zod-validálás — a séma alakot köt, nem igazságot.
- Streaming a UI-ra; prompt-verziózás gitben — a prompt kód.

3 · Mik az embeddingek, és mire jók?

- Szöveg (vagy kép) sűrű vektorreprezentációja, ahol a szemantikai hasonlóság vektortávolság (koszinusz-hasonlóság).
- Használat: szemantikus keresés, RAG-retrieval, dedup/klaszterezés, ajánlás.
- Infrastruktúra: vektorindex — pgvector kézenfekvő, ha már Postgresen vagy; nagy skálán dedikált vektor-DB.
- Gyakorlat: a chunkolás és az embedding-modell megválasztása többet számít, mint az index típusa.

4 · Magyarázd el a RAG-ot elejétől a végéig.

A releváns adatot lekérdezéskor teszed a kontextusba, ahelyett hogy a modell „tudná”.

- Pipeline: dokumentum-betöltés → struktúra-tudatos chunkolás átfedéssel → embedding → vektorindex → kérdés-embedding → top-k retrieval (ideálisan hibrid: vektor + kulcsszó, majd rerank) → válasz a chunkokra alapozva, hivatkozásokkal.
- Nyereség: friss adat, retrieval-kori hozzáférés-kontroll, visszakövethető válasz.
- Hibamódok: rossz chunkolás, retrieval-tévesztés, kontextus-tömés — a szűk keresztmetszet tipikusan a retrieval, nem a modell.

5 · RAG vagy fine-tuning?

- RAG a *tudáshoz*: friss, tenantonkénti, auditálható, olcsón frissül (újraindex, nem újratanítás).
- Fine-tuning a *viselkedéshez/formához*: konzisztens hangnem, szűk strukturált feladat volumenben.
- Sorrend: prompting + RAG; fine-tune csak eval-lal bizonyított, tartós részre, elég minőségi adattal.

6 · Hogyan kezeled a hallucinációt?

- Grounding: „csak a megadott forrásokból válaszolj, különben mondd, hogy nem tudod”.
- Séma-korlátozott kimenet; gépi kimenetek validálása a valóság ellen (létezik-e a szállítmány-ID).
- Hivatkozások a felhasználónak; emberi jóváhagyás minden költséges akción.
- Mérés: eval-készlet ismert Q&A-párokból minden prompt/modell-változásra — a vibe nem regressziós teszt.

7 · Hogyan működik a tool/function calling?

- A modellnek függvényeket írsz le: név + leírás + JSON-séma paraméterek.
- A modell nem futtat: strukturált „hívd X-et ezekkel az argokkal” üzenetet ad; a *te kódod* validál, végrehajt, visszaadja az eredményt; a modell folytatja — ciklus a végső válaszig.
- Minden a séma/leírás minőségén múlik; a modell kimenete nem megbízható input — végrehajtás előtt mindig validálsz.

8 · Mi az MCP, és hogyan épül fel?

Model Context Protocol — nyílt szabvány arra, hogy AI-kliensek egységesen kapcsolódjanak toolokhoz és adatforrásokhoz (Anthropic indította 2024 végén, azóta széles ökoszisztéma).

- Szereplők: host/kliens (pl. egy asszisztens-app) ↔ MCP **szerver**, JSON-RPC fölött.
- Primitívek: *tools* (hívható műveletek), *resources* (olvasható adatok), *prompts* (sablonok).
- Transzportok: stdio lokálisan, streamelhető HTTP távoli szerverhez; capability-egyeztetés a kapcsolat elején.
- Tervezési kérdések: jó tool-séma és leírás; idempotens, biztonságosan retryelhető toolok; auth és jogosultság-szűkítés a szerveroldalon.

9 · Mikor építs agentet — és mikor ne?

- Létra: prompt → láncolás → routing → párhuzamosítás → orchestrator-workerek → evaluátor-hurok → autonóm agent; minden fok képességet ad, kiszámíthatóságot vesz el.
- Ismert munkafolyamat → workflow LLM-lépésekkel: determinisztikus, tesztelhető.
- Valódi agent (a modell választ toolt, iterál) csak nyílt végű feladatra — és akkor is: iterációs limit, allowlistelt toolok, költségplafon, emberi jóváhagyás a mellékhatásokra.
- Alapelv: a legegyszerűbb működő megoldás; autonómia csak akkor, ha a feladat kikényszeríti.

10 · Éles üzemi szempontok LLM-funkcióknál?

- Latencia: token-streaming; modell-routing (egyszerű → kicsi, nehéz → nagy); prompt/prefix cache.
- Költség: token-büdzséként, kéresekénti költség-monitor, kontextus-vágás.
- Megbízhatóság: timeout, retry backoffal, fallback modell/szolgáltató, circuit breaker — szélsőséges távoli függőségként kezelve.
- Minőség: verziózott promptok, eval-suite CI-ban, canary a prompt/modell-váltásra, visszajelzés-gyűjtés.
- Megfelelés (EU/GDPR): adatfeldolgozási megállapodás, minimalizált PII a promptban, megőrzési szabályok.

11 · Prompt injection — mi az, és hogyan védekezel?

- Nem megbízható tartalomban (e-mail, dokumentum, weboldal) utasítás, amit a modell követhet — „hagyd figyelmen kívül a szabályaidat...”.
- Szűréssel nem oldható meg teljesen → architektúrával: megbízható utasítás és nem megbízható adat szigorú szétválasztása.
- Least-privilege, allowlistelt toolok (az összefoglalónak nincs e-mail-küldője); emberi megerősítés a következményes akciókra.
- Kimenet-validálás, semmi secret a kontextusban, audit-log.
- Mentális modell: okos gyakornok, aki ellenséges leveleket olvas — segít, de pénzt nem utalhat.

9 · **Git, CI/CD, Docker**

A „hogyan szállítotok?” kérdésblokk.

1 · **Branching- és PR-kultúra?**

- Trunk-based, rövid életű branchek; kis PR-ek — percek alatt átnézhető, egyben revertelhető.
- Nem user-kész funkció feature flag mögé merge-ölve; GitFlow csak valóban lassú release-vonatokhoz.
- Review: gyors átfutás; helyesség és érthetőség a téma — a stílus a linteré; a CI nem alku tárgya.

2 · **Rebase vagy merge?**

- Rebase: lineáris történet, tiszta bisect — a saját, még nem publikált branchedre.
- Aranyszabály: publikus/közös branchet nem rebase-elsz — másnak széttöröd a történetét.
- PR-be érkezés: squash merge (egy logikai változás = egy commit) vagy merge commit, csapatkonvenció szerint.
- Említhető még: `git bisect` hibakereséshez, `cherry-pick` hotfix backporthoz.

3 · Hogy néz ki egy jó CI-pipeline TS-repóhoz?

- Minden PR-en: cache-elt install → lint → `tsc --noEmit` → unit/integrációs tesztek → build → (szolgáltatásnál) image-build egyszer, commit SHA taggel.
- A gyors visszajelzés feature: párhuzamosítás, cache, percekben mérhető teljes futás.
- Ugyanaz az immutábilis artifact promótálódik stagingen át prodba — környezetenként soha nem épül újra.

4 · Deploy-stratégiák és rollback?

- Rolling a rutinra; blue/green az azonnali váltás-visszaváltásra; canary (kis forgalom-% + metrikafigyelés) a kockázatosra.
- Feature flag: a deploy és a release szétválasztása — a rollback gyakran egy kapcsoló.
- Az örök csapda a DB: expand-contract migráció — új/nullable mező mellé, backfill, olvasás átkapcsolása, eltávolítás később; így az ablakban bármely app-verzió működik, és a rollback biztonságos.

5 · Docker Node-hoz — a lényeg?

- Multi-stage build: deps + build külön, a runtime image-be csak a production-artifactok.
- Non-root user; exec-form CMD + SIGTERM-kezelés, hogy a graceful shutdown működjön.
- Healthcheck-endpoint, `.dockerignore`, pinnelt base image-ek.
- Kis image = gyorsabb deploy, kevesebb CVE.

6 · Docker layer-cache — hogyan hozol ki belőle gyors buildet?

- A rétegek fentről lefelé cache-elődnek: előbb `COPY package*.json + install`, csak utána a forráskód — így a kódváltozás nem invalidálja a dependency-réteget.
- `.dockerignore`: ne kerüljön be `node_modules`, `.git` — kisebb kontextus, stabilabb cache.
- CI-ban: registry-alapú cache (`--cache-from` / `buildx cache export`) a futók között.

10 · Web-biztonság

Full-stack szerepben a frontend- és backend-oldali támadások is a te asztalod.

1 · XSS — típusok és védekezés Reactben?

- Lényeg: támadói script fut a te oldalad kontextusában → session/token-lopás, akciók a user nevében.
- Típusok: stored (adatbázisból jön vissza), reflected (URL-paraméterből), DOM-alapú (kliensoldali sink).
- React alapból escape-eli az interpolált szöveget; a veszélyzóna: `dangerouslySetInnerHTML` — csak szanitizálva (`DOMPurify`), és URL-sinkek (`javascript: href`).
- Mélységi védelem: CSP, `httpOnly` cookie-k (a token ne legyen JS-ből olvasható).

2 · CSRF — mikor fenyeget, és mi véd ellene?

- A böngésző automatikusan küldi a cookie-t → egy idegen oldal a bejelentkezett user nevében süthet el state-változtató kérést.
- Csak cookie-alapú authnál releváns; Authorization headeres API-t nem érint.
- Védelem: `SameSite=Lax/Strict` cookie-k + CSRF-token a state-változtató kérésekre; a GET soha ne változtasson state-et.

3 · CORS — mi történik valójában?

- Böngésző-oldali védelem, nem server-auth: cross-origin válasz csak akkor jut el a JS-ig, ha a server `Access-Control-Allow-Origin` (stb.) headerekkel engedi.
- „Nem egyszerű” kérés előtt preflight OPTIONS megy (metódus/headerek egyeztetése).
- Credentials mód: `Allow-Credentials: true` kell, és az origin nem lehet `*`.
- Gyakori félreértés: CORS-hiba $\neq$ a server nem kapta meg; Postmanból „működik”, mert nincs böngésző.

4 · Hol tárolod a JWT-t a böngészőben?

- `localStorage`: kényelmes, de XSS-sel lopható.
- `httpOnly` + `Secure` + `SameSite` cookie: XSS ellen védett, cserébe CSRF-védelem kell — összességében ez a jobb default.
- Mérséklés mindkét úton: rövid életű access token + rotált refresh; SPA-nál gyakori a BFF-minta — a token a serveren marad, a böngésző session-cookie-t kap.

11 · **Tesztelés**

A React-specifikus tesztelés a 3. fejezetben; itt az általános kérdések.

1 · Hogyan osztod el a teszteket (piramis)?

- Sok gyors unit-teszt a tiszta logikára; erős integrációs réteg a valódi határokkal (DB, HTTP) — full-stack appnál gyakran ez adja a legtöbb bizalmat; vékony E2E a kritikus üzleti utakra.
- A cél nem a lefedettség száma, hanem: „merek-e péntek délután deployolni”.
- Integrációs teszthez valódi függőség konténerben (pl. Testcontainers-jellegű Postgres), nem átmockolt adatbázis.

2 · Mit mockolsz, és mit nem?

- Mockolod a rendszerhatárt: külső API-k, idő, véletlen, e-mail-küldés — amit nem birtokolsz, vagy nem determinisztikus.
- Nem mockolod a saját belső moduljaidat rétegről rétegre — az implementációhoz ragasztja a tesztet, és refaktornál hamisan piros/zöld.
- Hálózatot a határon mockolj (MSW/nock), ne a saját kliensfüggvényedet.

3 · Flaky tesztek — okok és kezelés?

- Tipikus okok: időzítésre várás fix sleep-pel, tesztek közti megosztott state/adat, sorrend-függés, valódi hálózat, nem várt aszinkron.
- Kezelés: feltételre várás (retry/poll) sleep helyett; izolált, saját adatot előállító tesztek; determinisztikus idő/random injektálva.
- Folyamat: a flaky teszt azonnal karanténba + jegy — a „néha piros” suite gyorsan elveszti a hitelét.

12 · Rendszertervezés, logisztikai ízzel

Módszer mindig: követelmények → API → adatmodell → skálázási út → hibamódok → observability. A trade-offokat hangosan.

1. szcenárió — Valós idejű szállítmánykövetés

- **Tisztázás:** aktív szállítmányok száma, frissítési gyakoriság, mennyire friss a „valós idejű” (2 mp vs 2 perc mindent átír), web/mobil kliensek.
- **Beérkezés:** pozíciók API-n/webhookon → validálás → queue mint burst-puffer.
- **Tárolás:** legutóbbi pozíció szállítmányonként Redisben (forró olvasás); teljes history append-only Postgres/idősoros táblába.
- **Push:** SSE (egyirányú, proxy-barát) vagy WebSocket (kétirányú); fan-out Redis pub/subbal a példányok közt.
- **Hibamódok:** stateless socket-réteg horizontálisan; reconnect last-event-id-vel; degradálás pollingra; out-of-order pozíciók dedupja timestamp szerint.

2. szcenárió — Azonnali ajánlat / árazó szolgáltatás

- **Tisztázás:** inputok (útvonal, súly, jármű, sürgősség), latencia-cél ($p_{95} < \sim 300$ ms?), az ajánlat érvényességi ablaka, audit-igény.

- **Terv:** stateless ajánlat-API; árazás = díjtáblák + felárak; referenciaadatok memória/Redis cache-ben TTL + verziózott invalidálással; minden kiadott ajánlat perzisztálva inputokkal + díjverzióval.
- **Peremesetek:** díjtábla-frissítés menet közben → konzisztens snapshot-verzióval árazol; külső távolság-szolgáltató kiesése → fallback-mátrix, becslésként jelölve; duplikált beküldés → idempotencia-kulcs.
- **Evolúció:** ajánlat→foglalás konverzió logolása (későbbi ML-árazás adata); a szabálmotor üzleti oldalról hangolható deploy nélkül.

3. szcenárió — Partner-webhook fogadás skálán

A 7. fejezet mini-architektúrája (API GW → ellenőrzés → SQS → idempotens fogyasztó → Postgres → EventBridge → DLQ). Hangsúlyok: gyors válasz + aszinkron feldolgozás; dedup event ID szerint; poison-message izoláció; partnerenkénti rate-izoláció, hogy egy zajos fél ne éheztesse a többi; visszajátszhatóság a tárolt nyers eseményekből.

1 · WebSocket vs SSE vs polling?

- Polling: legegyszerűbb, terheléses és késleltett — fallbacknek jó.
- SSE: egyirányú szerver→kliens stream HTTP fölött; auto-reconnect + last-event-id; proxy-barát — státuszfrissítésekre gyakran elég.
- WebSocket: kétirányú, alacsony overhead — chat, kollaboráció, nagy frekvenciájú telemetria.
- Skálázásnál mindkét pushnál: stateless kapcsolatkezelő réteg + Redis pub/sub fan-out.

2 · Cache-stratégiák és invalidálás?

- Szintek: böngésző/CDN → alkalmazás (Redis) → DB-oldali (materialized view).
- Alapminta: cache-aside — olvasás cache-ből, miss esetén DB + visszaírás TTL-lel.
- Invalidálás: TTL mint védőháló + eseményvezérelt törlés íráskor; verziózott kulcsok (`ra-tes:v42`) a „mindent egyszerre” váltáshoz.
- Stampede ellen: rövid lock / single-flight, korai frissítés, TTL-jitter.

3 · Rate limiting — algoritmusok és elhelyezés?

- Token bucket: átlagráta + burst-tűrés — a tipikus jó default; sliding window a precízebb számláláshoz; fixed window a legolcsóbb, hatéffektussal.
- Elosztottan: Redis-alapú számlálók; kulcs per user / API-key / IP, végpontonként eltérő limittekkel.
- Válasz: 429 + `Retry-After` ; a kliensoldal backoffal + jitterrel retryel.
- Hely: edge/gateway az olcsó elutasításhoz + kritikus belső függőségek előtt védelemként.

4 · Mi a transactional outbox, és miért kell?

- Probléma (dual write): DB-írás + esemény-publikálás két rendszerbe — bármelyik elbukhat a másik után, inkonzisztencia.
- Megoldás: az esemény ugyanabban a DB-tranzakcióban egy outbox-táblába kerül; külön relay olvassa és publikálja a brokerre.
- Következmény: at-least-once kézbesítés → a fogyasztó deduplikál event ID szerint.

5 · Saga — hogyan kezelsz több szolgáltatáson átívelő folyamatot?

- Elosztott tranzakció helyett: lokális tranzakciók sora + kompenzáló műveletek hibaágra (foglálás törlése, visszatérítés).
- Koreográfia: a szolgáltatások egymás eseményeire reagálnak — laza csatolás, nehezebben átlátható folyamat.
- Orchestráció: központi vezérlő lépteti — explicit állapotgép, jobb láthatóság, plusz komponens.
- Mindkettőnél: idempotens lépések, timeout + retry, a folyamat állapota lekérdezhető.

6 · Konzisztencia elosztott rendszerben — hogyan gondolkodsz róla?

- CAP-lényeg: hálózati partíció alatt konzisztencia vagy elérhetőség közül választasz.
- Gyakorlat: egy szolgáltatáson belül erős konzisztencia (Postgres-tranzakciók); szolgáltatások közt eventual consistency eseményekkel.
- Az eventual-t elviselhetővé teszi: idempotencia, sorrend-kezelés (verzió/timestamp), kompenzáció, és a UI őszintesége („feldolgozás alatt”).

7 · Monolit vs microservice-ek?

- Jól modularizált monolit a helyes default: egy deploy, egy tranzakcióhatár, szabad refaktor.
- Kivágás konkrét kényszerre: független skálázási profil, eltérő runtime/hibaizoláció, csapattulajdonlási határ.
- A microservice a kódkomplexitást elosztott-rendszer-komplexitásra cseréli: eventual consistency, saga, observability — ezt az árat tudatosan kell vállalni.

13 · PHP és legacy-migráció

A hirdetés „plusz” pontja — a kérdések itt jellemzően a migrációs gondolkodást mérik.

1 · Hogyan migrálsz el egy legacy PHP-rendszerről az üzlet leállítása nélkül?

- Strangler fig: először a legacy megfigyelhetővé és biztonságossá tétele (log, monitoring, tesztek a kritikus flow-k köré).
- Routing-homlokzat (gateway/proxy) elé — a forgalom endpointonként terelhető.
- Kiemelés képességként TS-szolgáltatásokba, a legnagyobb fájdalom/érték elsőnek; anti-corruption layer fordítja a legacy modelleket tiszta domain-kontraktusokra.
- Az adat a nehéz rész: képességként tiszta tulajdonlás, szinkron eseményekkel — hosszú távú közös-DB csatolás helyett.
- Endpoint-nyugdíjazás mért nulla használatnál; soha nem big-bang.

2 · PHP ↔ Node interop-minták?

- REST vagy queue a két világ közt, explicit kontraktussal (OpenAPI belső API-ra is).
- Események kifelé a PHP-ből: outbox-tábla polozva/hookolva — az új szolgáltatások szinkron csatolás nélkül reagálnak.
- Idempotens fogyasztók (retry van); correlation ID mindkét stacken át, hogy egy trace mesélje a teljes utat.
- Két író egy táblára: tilos — ott halnak meg a migrációk.

3 · Modern PHP — mit érdemes tudni róla?

- Ökoszisztéma: Composer + autoloading, PSR-szabványok, Symfony/Laravel táj.
- PHP 8.x: típusos property-k, enumok, attribútumok, match, constructor promotion, fiberek.
- Futtatási modell: hagyományosan request-per-process (FPM) — a Node hosszú életű processzével szemben; ez magyarázza a state- és pool-különbségeket.

14 · Puska

Egysorosok, közvetlenül az interjú előttre.

- **Böngésző-loop:** task után a teljes microtask-sor kiürül → ezért előzi a `then` a `setTimeout`-ot.
- **Node-loop sorrend:** szinkron kód → `process.nextTick` → promise-microtaskok → timers → poll (I/O) → `setImmediate`.
- **this:** new → explicit bind → metódushívás → sima hívás (strict: undefined); az arrow lexikális.
- **Hook-szabályok:** top level, azonos sorrend; effect = külső rendszer szinkronizálása, cleanup kötelező.
- **Memo-trió:** React.memo skippeli a gyereket; `useCallback/useMemo` stabil referenciát ad, hogy a memo működjön.
- **Idempotencia:** retry elkerülhetetlen ⇒ POST-hoz Idempotency-Key; queue/webhook-fogyasztó event ID szerint deduplikál.
- **Státuszkódok:** 201 létrehozva · 202 async · 204 üres · 401 ki vagy? · 403 nem a tiéd · 409 állapotütközés · 422 érvénytelen · 429 lassíts.
- **ETag-minta:** olvasás If-None-Match → 304; írás If-Match → 412 elavult verzióra.
- **Izolációs szintek:** RC (PG default) → RR (snapshot) → Serializable (retry-re készülj); anomáliák: dirty / non-repeatable / phantom.
- **Index:** B-tree, bal-prefix a composite-nál; írásköltség; EXPLAIN ANALYZE a bizonyíték.
- **SQS:** visibility timeout, DLQ + riasztás; FIFO = sorrend csoportonként, kisebb áteresztés.

- **DynamoDB:** access patternre tervezz; PK+SK; GSI a második kérdésre; mindenhol idempotens írás.
- **Cache:** cache-aside + TTL védőháló + eseményvezérelt invalidálás; stampede ellen single-flight + jitter.
- **Rate limit:** token bucket = ráta + burst; 429 + Retry-After; elosztva Redis-számlálóval.
- **Outbox:** esemény a DB-tranzakcióban, relay publikál; dual write soha.
- **CORS:** böngésző-védelem, nem auth; preflight = OPTIONS; credentials mellett nincs *.
- **JWT-tárolás:** httpOnly+SameSite cookie > local-Storage; rövid access + rotált refresh.
- **MCP-primitívek:** tools / resources / prompts JSON-RPC fölött; stdio lokálisan, streamelhető HTTP távolra.
- **Agent-létra:** prompt → lánc → routing → párhuzam → orkestráció → eval → autonóm; csak ameddig a feladat kényszerít.
- **Migrációs mantra:** strangler fig, anti-corruption layer, expand-contract az adatra; mérj, aztán nyugdíjazz.